

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2
1ra práctica (tipo b)
Primer Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales**, **NI OBJETOS** (con excepción de los elementos de `iostream`, `omanip` y `fstream`). **NO PUEDE UTILIZAR LA CLASE string**. Tampoco se podrán emplear las funciones `malloc`, `realloc`, `memset`, `strtok` o `strdup`, igualmente no se puede emplear cualquier función contenida en las bibliotecas `stdio.h`, `cstdio` o similares y que puedan estar también definidas en otras bibliotecas. **NO PODRÁ EMPLEAR PLANTILLAS EN ESTE LABORATORIO**
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN Estricto DISEÑO DESCENDENTE. Cada función **NO debe sobrepasar las 20 líneas de código aproximadamente**. El archivo `main.cpp` solo podrá contener la función `main` de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo `main.cpp` deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final**.
- El código comentado **NO SE CALIFICARÁ**. De igual manera **NO SE CALIFICARÁ** el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestren resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.

SE LES RECUERDA QUE, DE ACUERDO CON EL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

- **Puntaje total: 20 puntos.**

INDICACIONES INICIALES

Cree un proyecto de C++ en CLION siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"Lab01_2026_1_CO_PA_PN"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará los proyectos solicitados en la prueba.**
- En cada proyecto que desarrolle deberá crear una carpeta denominada **"Bibliotecas"** y allí coloque la biblioteca de funciones que necesite. Dentro de la carpeta **"cmake-build-debug"** cree las carpetas **"ArchivosDeDatos"** y **"ArchivosDeReporte"** y allí respectivamente coloque el archivo de datos que se le proporcionará y el archivo del reporte solicitado en la prueba. De no colocar esto se descontará un punto en cada proyecto.

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 1 del curso: "Funciones y alcance de variables". En este laboratorio se desarrollará una **biblioteca estática de funciones** en la que se implementen sobrecargas de operadores y funciones que le permitan solucionar el

problema planteado.

En la carpeta solicitada anteriormente, cree **dos carpetas** denominadas **“CrearBiblioteca_Parte1”** y **“UsarBiblioteca_Parte2”**, allí colocará los proyectos solicitados en las partes 1 y 2 respectivamente de este laboratorio. **DE NO COLOCAR ALGUNO DE ESTOS REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL. NO SE HARÁN EXCEPCIONES.**

PARTE 01 (10 puntos): CREACIÓN DE LA BIBLIOTECA ESTÁTICA

La clínica veterinaria **Huellitas** brinda servicios de atención médica para mascotas, principalmente perros y gatos. Debido al aumento en la cantidad de pacientes, la clínica desea implementar un pequeño sistema informático que permita gestionar las atenciones veterinarias de manera organizada.

En la clínica se registran mascotas con información relevante como su nombre, raza, color, tipo de animal (CANINO o FELINO) y fecha de nacimiento (aaaaammdd)

Cada mascota es atendida por veterinarios registrados en el sistema.

Una atención veterinaria corresponde a una cita programada en una fecha y hora específica con un veterinario. Durante la atención se pueden realizar distintos procedimientos como VACUNACIÓN, CONTROL o ESTERILIZACIÓN.

Existen algunas reglas importantes en la clínica:

1. Una mascota solo puede programar una esterilización si tiene al menos 6 meses de edad.
2. Un veterinario no puede atender dos mascotas al mismo tiempo, por lo que el sistema debe validar que el veterinario esté disponible en la fecha y hora solicitadas.

Primero, defina las estructuras necesarias para manejar la información descrita anteriormente:

struct Mascota que contiene idMascota, nombre, raza, color, tipo, fechaNacimiento (aaaaammdd).

struct Veterinario que contiene idVeterinario, nombre, especialidad.

struct Atencion que contiene idAtencion, idMascota, idVeterinario, fecha, tipoAtencion (VACUNACIÓN, ESTERILIZACIÓN, CONTROL), hora, minutos, estado (PROGRAMADA, CANCELADA)

Para guardar el **idAtencion** se genera un número correlativo que empiece en **1001**. Si existe el caso de la regla 2 dentro de las atenciones que se deseen guardar se pierde ese número correlativo.

struct SistemaHuellitas deberá permitir registrar nuevas atenciones y generar reportes de las atenciones realizadas que contiene **un arreglo** de struct Mascota, **un arreglo** de struct Veterinario, **un arreglo** struct Atencion, numeroMascotas, numeroVeterinarios y numeroAtenciones.

Se solicita que desarrolle una biblioteca estática en la cual se defina una serie de operadores sobrecargados que permitirá manejar las estructuras de datos creadas. Estas estructuras de datos están relacionadas con el registro de atenciones de las mascotas.

Las operaciones que la biblioteca estática permitirá realizar a través de sobrecargas de operadores se definen a continuación:

Lectura:

- Sobrecargando el operador **>>** de modo que permita leer los datos de **una** mascota de un archivo de tipo CSV. La operación **arch >> mascota;** involucrará una variable de archivo y una variable de tipo **“struct Mascota”**. Una línea de archivo tendrá la siguiente forma:

101,Luna,Labrador,Negro,CANINO,10/10/2024
(idMascota, nombre, raza, color, tipo, fecha de nacimiento)

- Sobrecargando el operador **>>** de modo que permita leer los datos de **un** veterinario de un archivo de tipo CSV. La operación **arch >> veterinario;** involucrará una variable de archivo y una variable de tipo **“struct Veterinario”**. Una línea de archivo tendrá la siguiente forma:

201,Miguel Perez,MedicinaGeneral

(idVeterinario, nombre, especialidad)

- Sobrecargando el operador >> de modo que permita leer los datos de una atención de un archivo de tipo CSV. La operación **arch >> atencion**; involucrará una variable de archivo y una variable de tipo **"struct Atencion"**. Una línea de archivo tendrá la siguiente forma:

```
01,204,7/4/2025,CONTROL,11:00,PROGRAMADA
(idMascota, idVeterinario, fecha, tipo, hora y minutos, estado
```

➤ **Operaciones:**

- Sobrecargando el operador == de modo que permita determinar si dos atenciones son iguales **atencionA == atencionB**; Si es igual debe devolver el valor **true**, si no es igual devolverá un valor **false**.
- Sobrecargando el operador <= de modo que permita comparar la fecha de nacimiento de una mascota con un valor entero en formato fecha aaaammdd. La operación **mascota <= fecha**; Si la fecha de nacimiento es menor o igual devolver **true**, si no es igual devolverá **false**.

➤ **Impresión:**

- Sobrecargando el operador << de modo que permita imprimir la información de una mascota. La operación **arch << mascota**; permitirá imprimir en un archivo de textos los datos contenidos en una variable de tipo **"struct Mascota"**. El formato será el siguiente:

```
ID: 101
Nombre: Luna
Raza: Labrador
Color: Negro
Tipo: CANINO
```

- Sobrecargando el operador << de modo que permita imprimir la información de una atención. La operación **arch << atencion**; permitirá imprimir en un archivo de textos los datos contenidos en una variable de tipo **"struct Atencion"**. El formato será el siguiente:

```
Fecha: 20250407, ID: 1002, Veterinario: 204, Tipo: CONTROL, Estado: PROGRAMADA
Fecha: 20260408, ID: 1003, Veterinario: 204, Tipo: VACUNACION, Estado: PROGRAMADA
...
```

Los datos deben aparecer con un formato adecuado y alineados correctamente, tanto para los enteros (hacia la derecha), valores de punto flotante (a la derecha y al punto decimal) y cadenas de caracteres (a la izquierda). **No se dará puntaje a esta parte si no respeta esta condición. NO SE HARÁN EXCEPCIONES.**

Consideraciones:

La solución debe contemplar la elaboración de:

- 1) un proyecto de implementación y prueba de las sobrecargas, denominado **"Fuentes_Biblioteca_2026_1"**, en este mismo proyecto genere la biblioteca estática (.a)
- 2) un proyecto donde se pruebe la biblioteca ya compilada **"Prueba_Biblioteca_2026_1"**. La prueba de las sobrecargas puede ser hechas lo más simple posible pero que muestre claramente que son correctas. Los dos proyectos deberán colocarse en la carpeta **"CrearBiblioteca_Parte1"**.

EN LAS PRUEBAS PARA LOS PROYECTOS 1 Y 2 SE PUEDE HACER DIRECTAMENTE EN LA FUNCIÓN MAIN, SE PUEDE HACER USO DE "HARD CODE" Y SE PUEDE SOBREPASAR LA CANTIDAD EL NUMERO DE LÍNEAS, SOLO EN ESTA PARTE DE LA PRUEBA.

EN LA BIBLIOTECA ESTÁTICA, NO PUEDE DESARROLLAR OTRAS FUNCIONES O SOBRECARGAS ADICIONALES A MENOS QUE ESTÉN DIRECTAMENTE RELACIONADAS A LAS SOBRECARGAS SOLICITADAS.

San Miguel, 10 de abril del 2026.